

Член-функции. Жизнен цикъл на обект. Конструктори и деструктори.

Капсулация. Модификатори за достъп.

Член-функции `get` и `set`.

`void f(int a);` → копиране променливата, не я променяме
`void f(int* &a);` → преназначаване на указател

член-функция - функции, които работят с член-данните на обекта

извиква се с обект на класа.

функция с уникално име и допълнителен параметър - константен указател към обекта

this

константни член-функции - не променят член-данните

`void g();` → `void g(A* const this)`

`void f() const;` → `void f(const A* const this)`

Конструктори и деструктор

• Конструктор - извиква се веднъж (при създаване на обекта)

- връща константна референция

- същото име като класа

- задава стойностите на член-данните

• Деструктор - извиква се веднъж (при изтриване на обекта)

• при излизане от scope-а, в който е деклариран

• при `delete[]/delete` върху динамични обекти

- няма тип на връщане

- ~ името на класа

Ако не са дефинирани, компилаторът създава конструктор и деструктор по подразбиране.

explicit — забранява на конструктора да е конвертиращ

```
struct A {  
    A(int x)  
    {  
    }  
};
```

explicit
→

```
struct A {  
    explicit A(int x)  
    {  
    }  
};
```

```
void f(A obj)  
{  
}
```

```
void f(const A& obj)  
{  
}
```

f(12) → създава временен
обект A

f(12) → compilation
error

A a(12);
f(a) → работи

При вложени обекти:

```
struct A  
{  
    A() { cout << "конструктор A "; }  
    ~A() { cout << "деструктор A "; }  
};
```

```
struct B  
{  
    B() { cout << "B() "; }  
    ~B() { cout << "B() "; }  
};
```

```
struct X  
{  
    A A first;  
    B array[3];  
    X() // A() B() B() B()  
    { cout << "X() "; }  
    ~X()  
    { cout << "X() "; }  
    ~X() ~B() ~B() ~B() ~A()  
};
```

```
int main()  
{  
    X obj; → X()  
    → ~X()  
}
```


Капсулация - "скриване на информация"

- разбива класа на интерфейс и имплементация
- определяме кои методи и атрибути може да използва потребителя на класа:

private → достъп в текущия клас

protected → достъп в класа и наследниците

public → достъп за всеки

struct е public

class е private

Селектори и мутатори - публични член-функции за достъпване (get) и промяна (set) на член-данни, които са private/protected

Mutable - позволява на член на обект да предефинира константността
- mutable на const обект не е const и може да се променя

```
struct A {  
    private:  
        mutable int n;  
    public:  
        void f() const {  
            n++;  
        }  
};
```

↓
все едно
вече не
е const

```
struct B {  
    private:  
        int n;  
    public:  
        void f() const  
        { n++; } // compilation error  
};
```

```
int main()  
{ const A a;  
  f(a) a.f(); → променя n,  
  независимо че f()  
  е const
```